

Exercícios — Aula 04

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Monte o projeto assim (você vai usá-lo de novo nas aulas 05, 06 e 07):

```
mkdir api-filmes && cd api-filmes
npm init -y
npm install express
```

No `package.json`, adicione `"type": "module"` e o script:

```
"scripts": { "dev": "node --watch server.js" }
```

Crie também um arquivo `api.http` para testar cada rota que você fizer.

Nível 1 — obrigatório

Exercício 1 — Servidor no ar

Em `server.js`, crie um servidor Express na porta `3000` com estas rotas:

Rota	Resposta
GET /	JSON com { nome, versao, autor } da sua API
GET /ping	JSON { "pong": true }
GET /hora	JSON com a hora atual (new Date().toISOString())

Confirme cada uma no navegador e pelo `api.http`.

Exercício 2 — Listando filmes

Coloque este array no topo do arquivo:

```
const filmes = [
  { id: 1, titulo: "Cidade de Deus", ano: 2002, genero: "drama", nota: 8.6, assistido: true },
  { id: 2, titulo: "Tropa de Elite", ano: 2007, genero: "acao", nota: 8.0, assistido: true },
  { id: 3, titulo: "Bacurau", ano: 2019, genero: "suspense", nota: 7.4, assistido: false },
  { id: 4, titulo: "Central do Brasil", ano: 1998, genero: "drama", nota: 8.0, assistido: false },
  { id: 5, titulo: "Auto da Compadecida", ano: 2000, genero: "comedia", nota: 8.6, assistido: true },
];
```

Crie:

- GET /filmes → devolve { total, dados }.
- GET /filmes/:id → devolve **um** filme.
 - Se o id não for número → 400 com mensagem clara.
 - Se o filme não existir → 404 com mensagem clara.

Exercício 3 — Filtros por query

Faça GET /filmes aceitar estes filtros (todos opcionais e **combináveis**):

Query	Efeito
?genero=drama	só daquele gênero
?assistido=true	só assistidos (cuidado: chega como texto!)
?notaMinima=8	só com nota maior ou igual
?busca=cidade	título contém o texto, ignorando maiúsculas

Teste combinações: /filmes?genero=drama¬aMinima=8.5

Nível 2 — esperado

Exercício 4 — Ordenação e limite

Adicione mais dois parâmetros de query:

- ?ordenarPor=nota → ordena por nota, ano ou titulo.
- ?ordem=desc → crescente (padrão) ou decrescente.
- ?limite=3 → devolve no máximo N filmes.

⚠ Lembre da aula 02: **sort** altera o array original. Copie antes com [...filmes].

Exercício 5 — Rotas de estatísticas

Crie:

- `GET /filmes/estatisticas/resumo` → `{ total, assistidos, naoAssistidos, notaMedia, anoMaisAntigo, anoMaisRecente }`
- `GET /filmes/estatisticas/por-genero` → `{ "drama": 2, "acao": 1, ... }`

⚠ **Atenção à ordem das rotas.** Se você declarar `/filmes/:id` antes destas, o Express vai entender "estatisticas" como um id. Declare as rotas fixas **antes** das rotas com parâmetro — e teste para ver o problema acontecer.

Desafio — opcional

Exercício 6 — Paginação

Implemente paginação de verdade em `GET /filmes`:

```
GET /filmes?pagina=2&porPagina=2
```

A resposta deve ter esta forma:

```
{
  "dados": [ ... ],
  "paginacao": {
    "paginaAtual": 2,
    "porPagina": 2,
    "totalItens": 5,
    "totalPaginas": 3,
    "temProxima": true,
    "temAnterior": true
  }
}
```

Dicas: `slice(inicio, fim)` corta o array; `inicio = (pagina - 1) * porPagina`; `Math.ceil` arredonda para cima. Valide que `pagina` e `porPagina` são números positivos.

Exercício 7 — Página HTML

Crie `GET /catalogo` que devolve uma **página HTML** (não JSON) com a lista de filmes montada em uma tabela, usando `map` e `join("")` para gerar as linhas.

Isso mostra que o servidor pode devolver qualquer coisa — HTML, JSON, texto. Nas aulas 08 e 09 vamos separar os papéis: o back devolve **só JSON** e o React cuida do HTML. Mas é bom ter visto que dá para fazer dos dois jeitos.

Checklist de entrega

- ☐ Todas as rotas respondem o status correto (200, 400, 404)
- ☐ Todo `res.status(...)` de erro tem `return` na frente
- ☐ Convertei `req.params.id` com `Number()` antes de comparar
- ☐ Lembrei que `req.query` sempre chega como texto
- ☐ Rotas fixas foram declaradas antes das rotas com `:parametro`
- ☐ Meu `api.http` tem um bloco para cada rota, incluindo os casos de erro